

Лабораторная работа №12

Операционные системы

Мустафина Аделя Юрисовна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Контрольные вопросы	14
6	Выводы	19
7	Список литературы	20

Список иллюстраций

4.1	1		8
4.2	2		9
4.3	3		10
4.4	4		10
4.5	5		11
4.6	6		12
4.7	7		13
4.8	8		13

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы.

2 Задание

1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.
2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.
3. Написать командный файл — аналог команды ls (без использования самой этой команды и команды dir). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.
4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

3 Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: — оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; — C-оболочка (или csh) — надстройка на оболочке Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд; — оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна; — BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation). POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна. Рассмотрим основные элементы программирования в оболочке bash. В других оболочках большинство команд будет совпадать с описанными ниже.

4 Выполнение лабораторной работы

Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из архиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку (рис. 4.1), (рис. 4.2).

A screenshot of a terminal window with a dark background. At the top, a light gray bar contains the text "GNU nano 8.1" on the left and "lab12-1.sh" on the right. The main area of the terminal shows the following text: a blue prompt "#!/bin/bash", a blue command "cp" followed by a yellow argument "\$@" and a yellow path "~/backup/" and a yellow filename "\${basename "\$@"}.bak", and a blue command "tar" followed by a yellow argument "-czf", a yellow path "~/backup/", a yellow filename "\${basename "\$@"}.tar.gz", and a yellow argument "\$@" followed by a blue cursor. A small white cursor is visible at the end of the last line.

```
GNU nano 8.1 lab12-1.sh
#!/bin/bash
cp "$@" ~/backup/"${basename "$@"}.bak"
tar -czf ~/backup/"${basename "$@"}.tar.gz" "$@"
```

Рис. 4.1: 1


```

aymustafina@aymustafina ~]$ touch lab12-1.sh
aymustafina@aymustafina ~]$ chmod +x lab12-1.sh
aymustafina@aymustafina ~]$ mkdir backup
mkdir: невозможно создать каталог «backup»: Файл существует
aymustafina@aymustafina ~]$ ls
abc1          Desktop      git-extended  LICENSE
australia     Documents   github.io     may
backup        Downloads   lab07.sh      monthly
backup_script.sh feathers    lab07.sh~     my_os
bin           file.txt    lab12-1.sh    '#new2.sh#'
aymustafina@aymustafina ~]$ cd backup
aymustafina@aymustafina backup]$ ls
aymustafina@aymustafina backup]$ cd ..
aymustafina@aymustafina ~]$ nano lab12-1.sh
aymustafina@aymustafina ~]$ bash lab12-1.sh
aymustafina@aymustafina ~]$ ls
abc1          Desktop      git-extended  LICENSE
australia     Documents   github.io     may
backup        Downloads   lab07.sh      monthly
backup_script.sh feathers    lab07.sh~     my_os
bin           file.txt    lab12-1.sh    '#new2.sh#'
aymustafina@aymustafina ~]$ cd backup
aymustafina@aymustafina backup]$ ls
lab12-1.sh.bak
aymustafina@aymustafina backup]$ nano lab12-1.sh
aymustafina@aymustafina backup]$ cd ..
aymustafina@aymustafina ~]$ nano lab12-1.sh
aymustafina@aymustafina ~]$ bash lab12-1.sh
aymustafina@aymustafina ~]$ cd backup
aymustafina@aymustafina backup]$ ls
lab12-1.sh.bak lab12-1.sh.tar.gz
aymustafina@aymustafina backup]$ 

```

Рис. 4.2: 2

Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов (рис. 4.3), (рис. 4.4).

```
GNU nano 8.1 lab12-2.sh
#!/bin/bash
for a in $(ls)
do echo $a
done
```

Рис. 4.3: 3

```
[aymustafina@aymustafina ~]$ touch lab12-2.sh
[aymustafina@aymustafina ~]$ ls
abc1      Desktop  git-extended  lab12-2.sh  '#new2.sh#'
australia Documents github.io     LICENSE     '#new3.sh#'
backup    Downloads lab07.sh      may         '#newfile1.'
backup_script.sh feathers  lab07.sh~    monthly     new-pass
bin       file.txt  lab12-1.sh   my_os       Pictures
[aymustafina@aymustafina ~]$ nano lab12-2.sh
[aymustafina@aymustafina ~]$ bash lab12-2.sh 1
1
[aymustafina@aymustafina ~]$ bash lab12-2.sh 1 2 3 4 5 6 7 8 9 10 11
1
2
3
4
5
6
7
8
9
10
11
[aymustafina@aymustafina ~]$
```

Рис. 4.4: 4

Написать командный файл — аналог команды `ls` (без использования самой этой ко- манды и команды `dir`). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога (рис. 4.5), (рис. 4.6).

```
GNU nano 8.1 lab12-3.sh
#!/bin/bash
for a in *
do
    if test -d "$a"
    then
        echo "$a^ is a directory"
    else
        echo -n "$a: is a file and "
        if test -w $a
        then
            echo writeable
            if test -r $a
            then
                echo "readable"
            else
                echo "neither readable or writeable"
            fi
        fi
    fi
done
```

Рис. 4.5: 5

```
[aymustafina@aymustafina ~]$ touch lab12-3.sh
[aymustafina@aymustafina ~]$ nano lab12-3.sh
[aymustafina@aymustafina ~]$ bash lab12-3.sh 1
abc1: is a file and writeable
readable
australia^ is a directory
backup^ is a directory
backup_script.sh: is a file and writeable
readable
bin^ is a directory
Desktop^ is a directory
Documents^ is a directory
Downloads^ is a directory
feathers: is a file and writeable
readable
file.txt: is a file and writeable
readable
git-extended^ is a directory
github.io^ is a directory
lab07.sh: is a file and writeable
readable
lab07.sh~: is a file and writeable
readable
lab12-1.sh: is a file and writeable
readable
lab12-2.sh: is a file and writeable
readable
lab12-3.sh: is a file and writeable
readable
LICENSE: is a file and writeable
readable
```

Рис. 4.6: 6

Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента ко-мандной строки (рис. 4.7), (рис. 4.8).

```

GNU nano 8.1 lab12-4.sh
#!/bin/bash
format=""
directory=""
echo "Формат файла"
read format
echo "Директория"
read directory
find "${directory}" -name "*${format}" -type f | wc -l
ls

```

Рис. 4.7: 7

```

[aymustafina@aymustafina ~]$ touch lab12-4.sh
[aymustafina@aymustafina ~]$ nano lab12-4.sh
[aymustafina@aymustafina ~]$ nano lab12-3.sh
[aymustafina@aymustafina ~]$ chmod +x lab12-4.sh
[aymustafina@aymustafina ~]$ bash lab12-4.sh
Формат файла
lab12-4.sh: строка 5: resd: команда не найдена
Директория
lab12-4.sh: строка 7: resd: команда не найдена
find: ': Нет такого файла или каталога
0
  abc1      Desktop    git-extended  lab12-2.sh   monthly
  australia Documents  github.io    lab12-3.sh   my_os
  backup     Downloads  lab07.sh     lab12-4.sh   '#new2.sh#'
  backup_script.sh feathers  lab07.sh~    LICENSE      '#new3.sh#'
  bin        file.txt   lab12-1.sh   may          '#newfile1.sh#'
[aymustafina@aymustafina ~]$ nano lab12-4.sh
[aymustafina@aymustafina ~]$ bash lab12-4.sh
Формат файла
pdf
Директория
/home/aymustafina
75
  abc1      Desktop    git-extended  lab12-2.sh   monthly
  australia Documents  github.io    lab12-3.sh   my_os
  backup     Downloads  lab07.sh     lab12-4.sh   '#new2.sh#'
  backup_script.sh feathers  lab07.sh~    LICENSE      '#new3.sh#'
  bin        file.txt   lab12-1.sh   may          '#newfile1.sh#'

```

Рис. 4.8: 8

5 Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются? Командная оболочка — это программа, которая предоставляет интерфейс для взаимодействия пользователя с операционной системой. Она позволяет пользователю вводить команды и получать результаты их выполнения. Примеры командных оболочек: Bash, Zsh, Ksh, Csh. Они отличаются синтаксисом, функциональностью, поддержкой расширенных возможностей (например, автозаполнение, управление историей команд) и поведением.
2. Что такое POSIX? POSIX (Portable Operating System Interface) — это стандарт, который определяет интерфейсы и поведение операционных систем, чтобы обеспечить совместимость между различными UNIX-подобными системами. Он включает в себя спецификации для командной строки, системных вызовов и библиотек.
3. Как определяются переменные и массивы в языке программирования bash? Переменные в Bash определяются простым присваиванием значения, например: `variable_name=value`. Массивы определяются с помощью синтаксиса: `array_name=(value1 value2 value3)`.
4. Каково назначение операторов `let` и `read`? Оператор `let` используется для выполнения арифметических операций, например: `let result=5+3`. Оператор `read` используется для чтения пользовательского ввода из стандартного ввода и присваивания его переменной, например: `read variable_name`.

5. Какие арифметические операции можно применять в языке программирования bash? В Bash можно применять основные арифметические операции: сложение (+), вычитание (-), умножение (*), деление (/), взятие остатка от деления (%).
6. Что означает операция (())? Операция (()) используется для выполнения арифметических вычислений и возвращает результат. Внутри скобок можно использовать арифметические выражения, например: ((result = a + b)).
7. Какие стандартные имена переменных Вам известны? Стандартные имена переменных в Bash включают: \$HOME (домашний каталог пользователя), \$USER (имя текущего пользователя), \$PATH (список каталогов для поиска исполняемых файлов), \$PWD (текущий рабочий каталог).
8. Что такое метасимволы? Метасимволы — это специальные символы, которые имеют особое значение в командной строке. Например, * (означает любое количество символов), ? (означает один любой символ), | (передает вывод одной команды на вход другой).
9. Как экранировать метасимволы? Метасимволы экранируются с помощью обратного слэша (). Например, чтобы использовать символ * как обычный символ, нужно писать *.
10. Как создавать и запускать командные файлы? Командные файлы создаются с помощью текстового редактора, например, nano или vim. Чтобы файл стал исполняемым, необходимо использовать команду chmod +x имя_файла. Запускать файл можно, введя его имя в терминале.
11. Как определяются функции в языке программирования bash? Функции в Bash определяются с помощью ключевого слова function, за которым следует имя функции и список команд в фигурных скобках. Например:

```
bash function my_function { # команды }
```

12. Каким образом можно выяснить, является файл каталогом или обычным файлом? Для проверки можно использовать команду `test` с флагом `-d` для каталогов и `-f` для обычных файлов. Например:

shell

```
bash test -d имя_файла && echo "Это каталог" test -f имя_файла && echo "Это файл"
```

13. Каково назначение команд `set`, `typeset` и `unset`?

- `set` — используется для установки значений переменных и параметров оболочки.
- `typeset` — определяет переменные и устанавливает их атрибуты (например, массивы).
- `unset` — удаляет переменные или функции.

14. Как передаются параметры в командные файлы? Параметры передаются в командные файлы через командную строку. Они могут быть доступны в скрипте с помощью символов `$1`, `$2`, ..., `$9`, а также `$@` (все параметры) и `$#` (количество переданных параметров). Например, если скрипт вызывается с командой `./script.sh arg1 arg2`, то внутри скрипта `arg1` будет доступен как `$1`, а `arg2` как `$2`.

15. Как использовать условные конструкции в Bash? Условные конструкции в Bash могут быть реализованы с помощью `if`, `elif`, и `else`. Например:

shell

```
bash if [ условие ]; then # команды, если условие истинно elif [ другое_условие ]; then # команды, если другое условие истинно else # команды, если ни одно из условий не истинно fi
```

16. Как организовать циклы в Bash? В Bash можно организовать циклы с помощью `for`, `while`, и `until`. Например, цикл `for` может выглядеть так:

shell

bash for item in список; do # команды для каждого элемента done

Цикл while будет выполняться, пока условие истинно:

shell

bash while [условие]; do # команды done

17. Как обрабатывать ошибки в Bash? Обработка ошибок в Bash может быть выполнена с помощью проверки кода возврата команд (с помощью \$?) или с использованием конструкции trap. Например:

shell

bash команда if [\$? -ne 0]; then echo “Ошибка выполнения команды” fi

18. Что такое “потoki” и как они используются в Bash? Потoki — это каналы, через которые происходит ввод и вывод данных. В Bash стандартные потоки включают стандартный ввод (stdin), стандартный вывод (stdout) и стандартный вывод ошибок (stderr). Их можно перенаправлять, используя символы >, », <, и 2>.

19. Как перенаправлять ввод и вывод в Bash? Перенаправление ввода и вывода осуществляется с помощью символов:

- для перенаправления стандартного вывода в файл (перезаписывает файл).
- для добавления стандартного вывода в конец файла.
- < для перенаправления ввода из файла.
- 2> для перенаправления стандартного вывода ошибок в файл.

20. Как создать и использовать массивы в Bash? Массивы в Bash создаются с помощью синтаксиса:

```
bash array_name=(value1 value2 value3)
```

Для доступа к элементам массива используется индекс:

```
shell
```

```
bash echo ${array_name[0]} # выводит первый элемент
```

Чтобы получить все элементы массива, можно использовать:

```
shell
```

```
bash echo ${array_name[@]}
```

6 Выводы

Изучила основы программирования в оболочке ОС UNIX/Linux. Научилась писать небольшие командные файлы.

7 Список литературы

[1]

1. Лабораторная работа №12.